Singleton

Season: 1 · Creational · Category: GoF · Level: Beginner · Build with Waqar

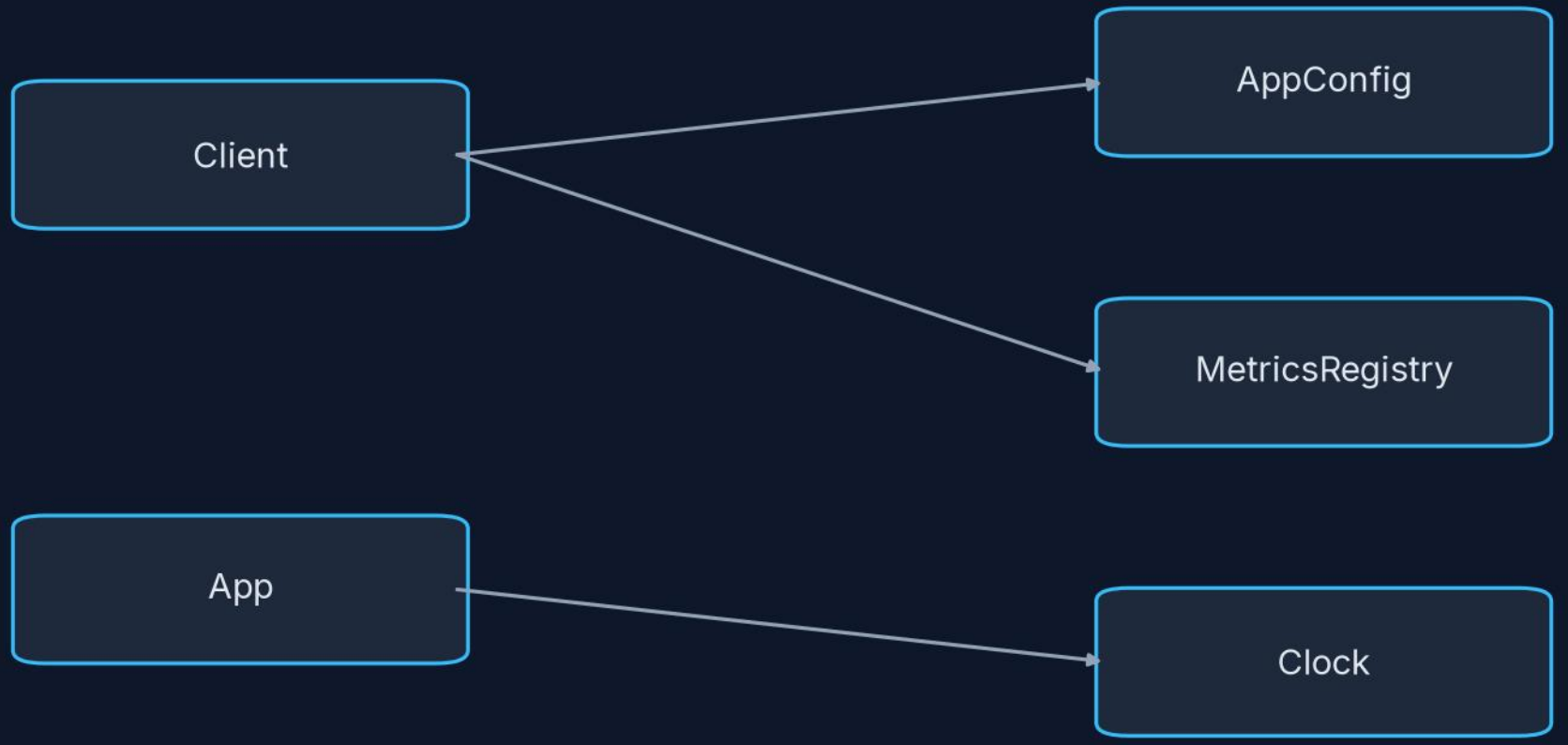
Today's Hook

- Today: Singleton as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Singleton structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Singleton
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/creational/singleton/AppConfig.java
package com.buildwithwaqar.patterns.gof.creational.singleton;

/** Preferred: enum singleton — serialization-safe, reflection-safe. */
public enum AppConfig {
    INSTANCE;

    private String env = "prod";

    public String env() { return env; }
    public void setEnv(String env) { this.env = env; }
    public String greeting() { return "hello from " + env; }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.creational.singleton;

import java.time.Instant;

/** Runnable walkthrough for the Singleton episode. */
public final class Demo {
    public static void main(String[] args) {
        System.out.println("=== Singleton walkthrough ===");
        AppConfig.INSTANCE.setEnv("staging");
        System.out.println("greeting=" + AppConfig.INSTANCE.greeting());
        System.out.println("sameEnum=" + (AppConfig.INSTANCE == AppConfig.INSTANCE));

        MetricsRegistry a = MetricsRegistry.getInstance();
        MetricsRegistry b = MetricsRegistry.getInstance();
        a.reset();
        a.hit(); a.hit(); a.hit();
        System.out.println("sameHolder=" + (a == b));
        System.out.println("hits=" + b.hits());

        Clock fixed = new Clock.FixedClock(Instant.parse("2026-01-01T00:00:00Z"));
        System.out.println("fixedNow=" + fixed.now());
    }
```

Demo.main (2)

```
}
```

Walkthrough Output

```
=== Singleton walkthrough ===
greeting=hello from staging
sameEnum=true
sameHolder=true
hits=3
fixedNow=2026-01-01T00:00:00Z
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Singleton
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Singleton → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.